

Inteligencia Artificial y Neurociencias

Licenciatura en Tecnología Digital, UTDT, 2S-2026

Profesores: Agustín Gravano, Andrés Rieznik

Auxiliares: Gerardo Del Toro, Antonella Giordano Furchi

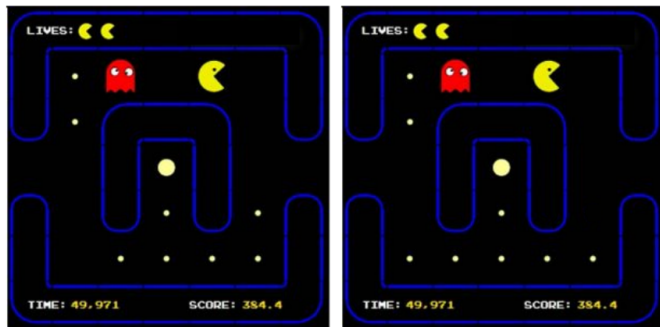
Deep Q-learning



Métodos tabulares

Los métodos tabulares estiman la función de valor en una [tabla \$Q\$](#) . Este enfoque no sirve cuando $\mathcal{S}$ es demasiado grande.

EJEMPLO: La cantidad de posibles imágenes en la pantalla de juegos como el PacMan o el Mario Bros es un número combinatorio gigantesco.



Estas 2 pantallas son [estados distintos](#), pese a que son muy similares.

Q-learning tabular

Inicializar $Q(s, a)$ arbitrariamente $\forall s \in \mathcal{S}, a \in \mathcal{A}(s)$

Repetir:

 Inicializar S

 Repetir:

$A \leftarrow$ acción desde S según política ε -greedy basada en Q

 Ejecutar la acción A ; observar R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 hasta que S sea terminal

Q-learning con aproximación de la función de valor

$\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$ es una función parametrizable
con pesos $\mathbf{w} \in \mathbb{R}^d$ (ej.: red neuronal)

Inicializar $\mathbf{w} \in \mathbb{R}^d$ arbitrariamente

Repetir:

 Inicializar S

 Repetir:

$A \leftarrow$ acción desde S según polít. ε -greedy basada en $\hat{q}(S, \cdot, \mathbf{w})$

 Ejecutar la acción A ; observar R, S'

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \max_a \hat{q}(S', a, \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$

$S \leftarrow S'$

 hasta que S sea terminal

Aproximación de la función de valor q

Vamos a aproximar $q_*(s, a)$ mediante $\hat{q}(s, a, \mathbf{w})$, una función con pesos $\mathbf{w} \in \mathbb{R}^d$, con $d \ll |\mathcal{S}|$.

EJEMPLO: $\hat{q}$ puede ser una red neuronal y $\mathbf{w}$ sus pesos.

Como $d \ll |\mathcal{S}|$, nuestra aproximación $\hat{q}$ tendrá cierto error.

Definimos el error cuadrático del valor estimado como:

$$\overline{\text{QE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}(s)} [q_*(s, a) - \hat{q}(s, a, \mathbf{w})]^2$$

Aproximación de la función de valor q

En un tiempo t , usamos la red con pesos $\mathbf{w}_t$ para estimar el valor de cada acción $A_t \in \mathcal{A}(S_t)$ en el estado $S_t \in \mathcal{S}$:

$$\hat{q}(S_t, A_t, \mathbf{w}_t) \approx q_*(S_t, A_t)$$

Supongamos que podemos observar una muestra de $q_*(S_t, A_t)$.

¿Cómo podríamos usar esa nueva información para mejorar nuestra aproximación?

Querríamos reducir $[q_*(S_t, A_t) - \hat{q}(S_t, A_t, \mathbf{w}_t)]^2$ (el error cuadrático de la estimación para S_t, A_t), y así reducir $\overline{\text{QE}}(\mathbf{w}_t)$.

Si $\hat{q}(s, a, \mathbf{w})$ es diferenciable con respecto a $\mathbf{w}$, podríamos usar el **descenso por el gradiente estocástico** (SGD) para ajustar $\mathbf{w}$ un poco en la dirección en que más se reduce el error.

Aproximación de la función de valor q

$$\begin{aligned}\mathbf{w}_{t+1} &\doteq \mathbf{w}_t - \alpha \nabla [q_*(S_t, A_t) - \hat{q}(S_t, A_t, \mathbf{w}_t)]^2 \\ &= \mathbf{w}_t + 2\alpha [q_*(S_t, A_t) - \hat{q}(S_t, A_t, \mathbf{w}_t)] \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)\end{aligned}$$

OBS.: Al '2' lo absorbe α : como es un parámetro arbitrario, da igual escribir α que 2α .

Muy lindo todo, pero en Q-learning no sampleamos q_* directamente...

Podemos usar la idea de *bootstrapping* para **estimar** q_* :

$$q_*(S_t, A_t) \approx R_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a, \mathbf{w}_t)$$

Así, llegamos a la regla de actualización con SGD:

$$\mathbf{w}_{t+1} \approx \mathbf{w}_t + \alpha \left[R_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a, \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t) \right] \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$$

Q-learning semi-gradiente para estimar q_* (DQL)

$\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$ es una función parametrizable
con pesos $\mathbf{w} \in \mathbb{R}^d$ (ej.: **red neuronal**)

Inicializar $\mathbf{w} \in \mathbb{R}^d$ arbitrariamente

Repetir:

 Inicializar S

 Repetir:

$A \leftarrow$ acción desde S según polít. ε -greedy basada en $\hat{q}(S, \cdot, \mathbf{w})$

 Ejecutar la acción A ; observar R, S'

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \max_a \hat{q}(S', a, \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$

$S \leftarrow S'$

 hasta que S sea terminal

En estos algoritmos no hay garantía de convergencia. Igual se los usa mucho. OBS.: Los algoritmos tabulares son casos particulares, y sí poseen buenas propiedades de convergencia.

Playing Atari with Deep Reinforcement Learning

Volodymyr Mnih Koray Kavukcuoglu David Silver Alex Graves Ioannis Antonoglou

Daan Wierstra Martin Riedmiller

DeepMind Technologies

{vlad,koray,david,alex.graves,ioannis,daan,martin.riedmiller} @ deepmind.com

LETTER

doi:10.1038/nature14236

Human-level control through deep reinforcement learning

Volodymyr Mnih^{1*}, Koray Kavukcuoglu^{1*}, David Silver^{1*}, Andrei A. Rusu¹, Joel Veness¹, Marc G. Bellemare¹, Alex Graves¹, Martin Riedmiller¹, Andreas K. Fiedelnd¹, Georg Ostrovski¹, Stig Petersen¹, Charles Beattie¹, Amir Sadik¹, Ioannis Antonoglou¹, Helen King¹, Dharmashan Kumaran¹, Daan Wierstra¹, Shane Legg¹ & Demis Hassabis¹

¹Google DeepMind, 5 New Street Square, London EC4A 3TW, UK.

*These authors contributed equally to this work.

Ejemplo: Atari con Deep Q-Learning: Mnih *et al.* 2013, 2015

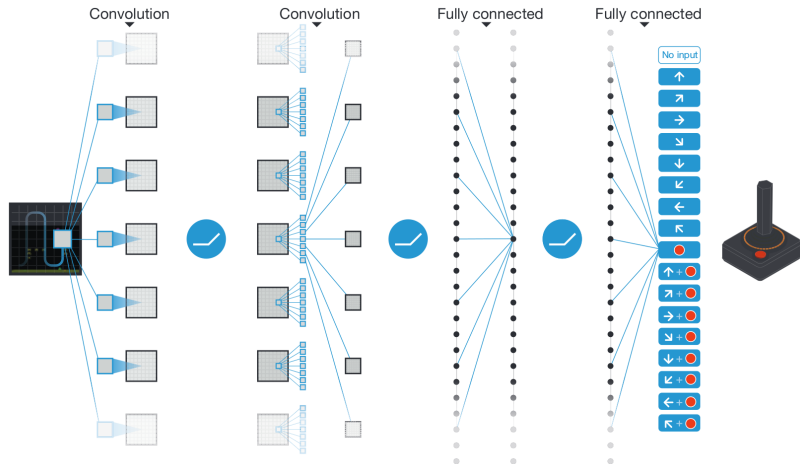
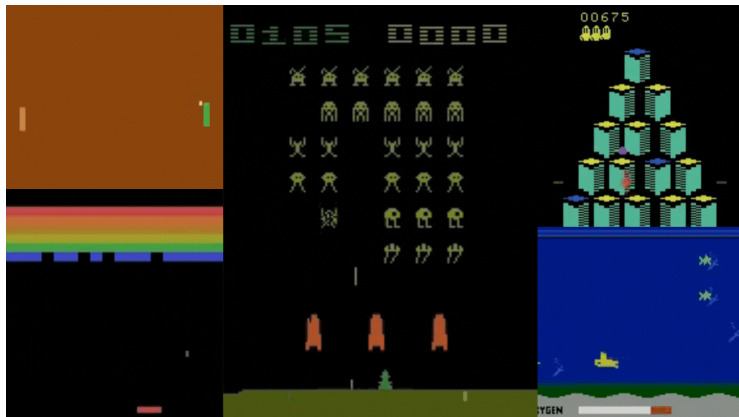


Figure 1 | Schematic illustration of the convolutional neural network. The details of the architecture are explained in the Methods. The input to the neural network consists of an $84 \times 84 \times 4$ image produced by the preprocessing map ϕ , followed by three convolutional layers (note: snaking blue line symbolizes sliding of each filter across input image) and two fully connected layers with a single output for each valid action. Each hidden layer is followed by a rectifier nonlinearity (that is, $\max(0, x)$).

DQL: Detalles importantes de implementación (Mnih *et al.* 2015)

- Técnica conocida como *replay de experiencias*:
 - Cada experiencia $e_t = (S_t, A_t, R_{t+1}, S_{t+1})$ observada por el agente se almacena en un conjunto $D_t = \{e_1, \dots, e_t\}$, que llamamos *memoria*.
 - Cada actualización de $\mathbf{w}$ se hace sobre un *minibatch* de experiencias sampleadas uniformemente del conjunto D_t .
- La red usada para estimar el valor objetivo (llamada *target network*) se actualiza cada cierta cantidad de iteraciones, copiando los pesos de la red principal (llamada *Q-network*).
- El tamaño de la memoria, el tamaño de los minibatches y cada cuántas iteraciones se actualiza la target network son *hiperparámetros* que se suman a α , ϵ y γ .

Deep Q-Learning sirve para aprender a jugar en Atari...



...y mucho más:

[Buscar en Google Scholar: “Deep Q-Learning”](#)